Enhancement Three Narrative: Databases

Artifact Overview

I selected Travlr Getaways, a full stack travel app developed using MEAN-stack technology as my artifact for the third enhancement. Travlr Getaways was originally designed in my Full Stack Development class. The primary components of the application include an express node.js api utilizing mongoDB through mongoose as its persistence layer, and an angular admin single page application that allows users to manage their trip records. The initial artifact included support for all of the above as well as viewing trip details, user registration/login, json web token based authentication, and crud operations for each record within a users trip history.

For this assignment, I used the same base application, but created a database focused version of the enhanced artifact at /artifacts/travlr-getaways-enhancement-three.

Rationale for inclucsion

Travlr Getaways fits into my ePortfolio because it is an entirely built-out full stack application rather than one-off isolated exercises. Therefore, the application shows all aspects of API design, authentication, routing, how to interact with the front-end, backend service architecture and MongoDB persistence. This will make this project a strong artifact for showing what type of backend and full-stack development projects I would like to work on as I am working professionally.

There were many ways to improve the database portion of the original artifact. Trip information was stored in MongoDB, however, significant values like price and duration were only stored as string representations of the value ($799 / 4 nights). Although these values displayed correctly within the User Interface, they did not provide much support for database query/validations/indexing/reporting. For instance, without numeric representation of the price/duration, there is no way for the database to perform reliable budget or duration based queries. Additionally, the original schema lacked the proper indexes and did not have normalized searchable text or timestamp based auditing for records.

Enhancement Three builds upon the existing artifact by building out a stronger MongoDB/Mongoose data model. In addition to storing the same display values for price/duration/etc., Enhancement Three creates additional normalized database fields for each of those values. The display values are still preserved but now the database has structured values that can be validated, indexed, queried. Timestamps were added along with upper-case normalized trip code values, schema validations, a composite index for budget and duration based queries and a text index for searchable trip content.

Enhancement and Evidence

Major database changes have been made to the file models/travlr.js in app_api/, repositories/trip-repository.js in app_api/, validators/trip-validator.js in app_api/, and utils/trip-normalization.js in app_api/.

The trip-normalization utility, which is a centralized tool for parsing and creating derived field logic, has been added. It will convert display prices (such as $799) into numeric representations (such as 799), parse durations (such as 4 nights) into a count of nights, normalize trip codes to all upper case letters, and create searchable text from the trip code, name, description, and details. Centralizing this logic makes it easier to maintain consistency across the API validator and Mongoose schema.

Mongoose schema now includes priceAmount, durationNights, and searchText. These are required fields that are created from existing display fields, validated prior to persistence, and represent numeric query fields. This provides an additional layer of data integrity for the database. In addition, since timestamps are defined in the schema, record creation and last updated time stamps may be tracked.

Additionally, the indexing strategy used by the database has been enhanced. A unique index on code, a composite index on priceAmount and durationNights, and a text index on searchText have been defined. These indexes are aligned with the application domain. Trip codes should be unique. Budget and duration provide a natural way to filter results for travel searches. Text search allows users to discover trips based upon their destinations or descriptions.

findForRecommendationCriteria() has been added to the repository layer. This function takes in normalized input from recommendation services and creates MongoDB query criteria. The database can then use these query criteria to pre-filter records based upon budget, duration, and searchable text prior to sending the pre-filtered list of candidate records back to the application level recommendation service. This ties together both enhancements without having the database duplicate the work of the application level recommendation service. While the algorithmic enhancement will continue to handle the ranking of candidate records and tie breaking, the database enhancement will improve how candidate records are stored and selected.

Additional automated tests have been added to validate the functionality of the database. These tests will check to see if the schema properly derives normalized fields, properly reject invalid values when they cannot populated numeric database fields, define proper indexes for uniqueness, filtering and text searching. Tests have also been added to verify that API payloads are being properly validated and build normalized fields before they are persisted to the database. Local testing using npm.cmd test passed all 23 backend tests: 23 total tests run; 23 total passed; 0 total failed.

Course Outcome Alignment

This enhancement directly supports Course Outcome 4 which is about showing ability to use solid techniques, skills and tools to implement solutions that deliver value and achieve specific industry goals. Enhancement uses schema design with Mongoose for validation, normalization of stored fields, indexes and timestamp logic to make travel app more reliable and useful. This enhancement also supports Outcome 5 because validation and predictable persistence are part of thinking security wise. Now malformed price and duration data is rejected rather than weak or unusable data being allowed into the database. While this isn't encryption or authentication change, it still reduces risk to data integrity and supports safer backend behavior. My plan for covering outcomes remains consistent with plan from Module One. Enhancement One showed strong evidence for software design and security. Enhancement Two showed direct evidence for algorithms and data structures. Enhancement Three now provides direct evidence for quality design of databases and persistence. Together these three enhancements show a broader path of improvement throughout the stack of artifacts.

Process, Challenges, and Learning

I learned a lot when I developed this part of the project. Database design involves much more than just choosing MongoDB and creating some simple CRUD commands. An application backed by a database requires data that is organized appropriately to address the questions that the application will need to answer. Although using string representations were sufficient to present the data as needed for the users to view the information, they were not sufficient to reliably filter, index, or analyze the data in any way. Normalizing those values into separate fields made the database more valuable to the project without changing anything about how the current user interface displays data.

A big problem was separating this enhancement from the algorithmic enhancements. Since the Recommendation Feature has already taken advantage of price and duration in the ranking logic, there was an opportunity to duplicate some of the work. However, I chose to focus this milestone on making sure that the data was persisted and ready for querying, while leaving all of the algorithmic logic for scoring and ranking to the Recommendation Service.

In addition to keeping these two enhancements separate, another large challenge was maintaining backward compatibility with the original application. I did not want to remove the original display fields since both the Public & Administrative User Interfaces currently utilize them. Therefore, I decided to keep Price and Duration in place for Display purposes, and add Price Amount and Duration Nights for Data Model Behavior. This was a reasonable trade off at the time as it provided an enhanced Data Model without necessitating a complete front-end rewrite.

Additionally, the testing process further emphasized the importance of knowing how frameworks behave. Initially, I had assumed that one Validation Path could derive Fields within each Test Case; however, I found out that the Synchronous Validation Path worked differently. As such, I modified the schema so that derived values can be accessed via defaults as well as validation logic. These changes greatly improved the reliability of the model and allowed me to better prove that the database enhancement worked as planned.